情報理工学実験 II

# ネットワークプログラミング


学部学科　：　工学部 情報エレクトロニクス学科

学年　：　学士 3 年

学生番号　：　02246910

学部番号　：　ICT26903

学生氏名　：　吉野 拓人

メールアドレス　：　takuto.yoshino.l3@elms.hokudai.ac.jp


提出日：2026 年 11 月 10 日

# 目次


目次 . . . . . 1

課題 A . . . . . 2

課題 B . . . . . 4

課題 C . . . . . 7

課題 D . . . . . 9

課題 E . . . . . 12

課題 F . . . . . 16

# 課題 A

## 課題内容

IPv4・IPv6 の相手に ICMP echo request を送り，応答の有無を確認した.

## 実験方法

1. 1 つ目のターミナルでキャプチャを開始

```
(IPv4) tcpdump -i eth0 -s 0 -w ping4.pcap
(IPv6) tcpdump -i eth1 -s 0 -w ping6.pcap
```

2. キャプチャ中に，別のターミナルで ping, ping6 を実行

```
(IPv4) ping -c 5 172.27.0.1
(IPv6) ping6 -c 5 www.docomo.ne.jp
```

3. ping の終了後，キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開く

## 結果

IPv4・IPv6 の tcpdump の実行結果と，ping・ping6 の実行結果をそれぞれ出力 1，出力 2，出力 3，出力 4 に示した.

出力 1: tcpdump(IPv4) の実行結果

```
guest@host82:~$ tcpdump -i eth0 -s 0 -w ping4.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C14 packets captured
14 packets received by filter
0 packets dropped by kernel
```

出力 2: tcpdump(IPv6) の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w ping6.pcap
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C132 packets captured
146 packets received by filter
0 packets dropped by kernel
```

出力 3: ping(IPv4) の実行結果

```
guest@host82:~$ ping -c 5 172.27.0.1
PING 172.27.0.1 (172.27.0.1) 56(84) バイトのデータ
64 バイト応答 送信元 172.27.0.1: icmp_seq=1 ttl=64 時間=0.029 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=2 ttl=64 時間=0.024 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=3 ttl=64 時間=0.050 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=4 ttl=64 時間=0.020 ミリ秒
64 バイト応答 送信元 172.27.0.1: icmp_seq=5 ttl=64 時間=0.026 ミリ秒

--- 172.27.0.1 ping 統計 ---
送信パケット数 5，受信パケット数 5，パケット損失 0%，時間 4097 ミリ秒
rtt 最小/平均/最大/mdev = 0.020/0.029/0.050/0.010 ミリ秒
```

出力 4: ping6(IPv6) の実行結果

```
guest@host82:~$ ping6 -c 5 www.docomo.ne.jp
PING www.docomo.ne.jp(2600:9000:2772:3600:7:a2b3:f240:93a1 (2600:9000:2772:3600:7:a2b3:f240:93a1)) 56 デー
↪  タ長 (byte)
^C
--- www.docomo.ne.jp ping 統計 ---
送信パケット数 5，受信パケット数 0，パケット損失 100%，時間 4100 ミリ秒
```

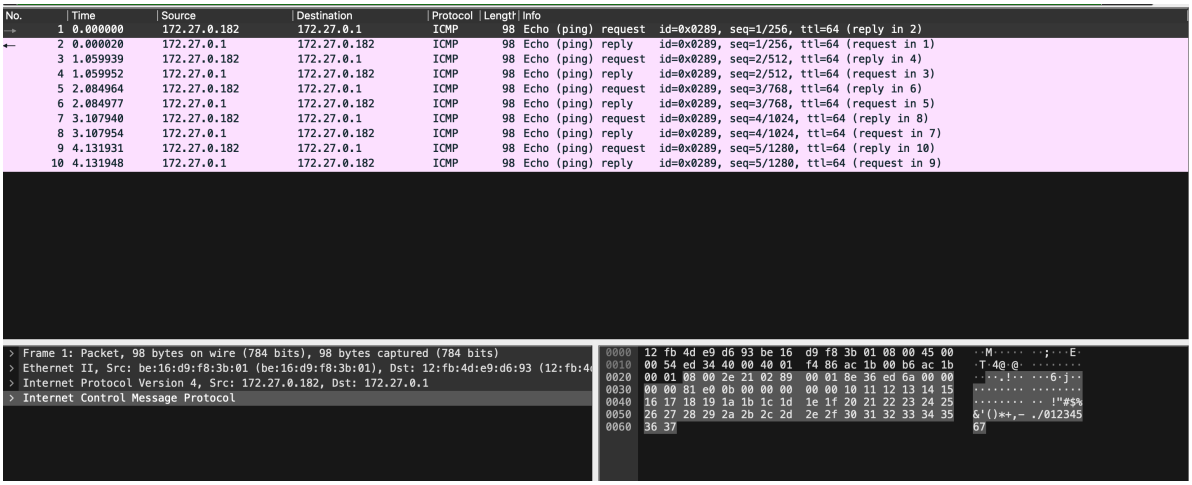



図 1: IPv4 の ping を Wireshark で読み取った結果

IPv4 では，送信元 172.27.0.182 から 172.27.0.1 に向けて ICMP echo request が 5 回送信され，それぞれに対する echo reply が確認できた．ping の結果も 5 パケット送信，5 パケット受信であり，パケット損失はなかった．平均往復時間は 0.029 ミリ秒であった．図 1 でも，request と reply が交互に並んでいることが確認できる．

IPv6 では www.docomo.ne.jp の名前解決はでき，2600:9000:2772:3600:7:a2b3:f240:93a1 宛ての echo request が 5 回送信されたが，echo reply は確認できなかった．ping の結果は 5 パケット送信，受信 0 であり，図 2 にも request のみが記録されていた．学外との ICMP 通信がネットワーク上で制限されている可能性があるため，この結果だけから通信相手の停止とは判断できない．

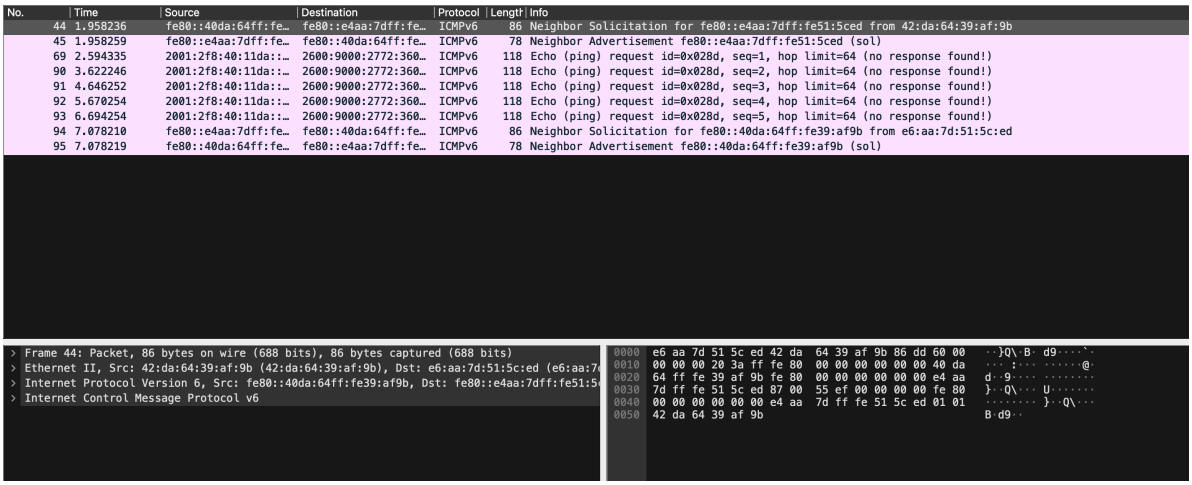



図 2: IPv6 の ping を Wireshark で読み取った結果

# 課題 B

## 課題内容

IPv4 では ARP，IPv6 では近隣探索で相手の MAC アドレスを調べ，そのパケットのやり取りを確認した．

## 実験方法

1. 実行前の近隣キャッシュを記録

```
ip ne
```

2. 1 つ目のターミナルでキャプチャを開始

```
tcpdump -i eth1 -s 0 -w B.pcap 'not port 4200'
```

3. キャプチャ中に，別のターミナルで arping，ndisc6 を実行

```
(IPv4) arping -c 3 172.28.0.200
(IPv6) ndisc6 2001:2f8:40:11da::200 eth1
```

4. 実行後に近隣キャッシュを再度記録

```
ip ne
```

5. キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開く

## 結果

tcpdump の実行結果，実行前後の近隣キャッシュ，arping と ndisc6 の実行結果をそれぞれ出力 5，出力 6，出力 7，出力 8，出力 9 に示した．

出力 5: tcpdump の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w B.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C498 packets captured
500 packets received by filter
0 packets dropped by kernel
```

出力 6: 実行前の近隣キャッシュ

```
guest@host82:~$ ip ne
172.28.0.203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
172.28.0.200 dev eth1 lladdr fe:89:01:c2:83:3a STALE
172.28.0.1 dev eth1 lladdr a2:1e:33:b9:98:ef REACHABLE
fe80::e4aa:7dff:fe51:5ced dev eth1 lladdr e6:aa:7d:51:5c:ed REACHABLE
fe80::a01e:33ff:feb9:98ef dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
2001:2f8:40:11da::1 dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
```

出力 7: arping(IPv4) の実行結果

```
guest@host82:~$ arping -c 3 172.28.0.200
ARPING 172.28.0.200 送信元 172.28.0.182 eth1
ユニキャスト 応答 送信元 172.28.0.200 [FE:89:01:C2:83:3A]  0.742 ミリ秒
ユニキャスト 応答 送信元 172.28.0.200 [FE:89:01:C2:83:3A]  0.522 ミリ秒
ユニキャスト 応答 送信元 172.28.0.200 [FE:89:01:C2:83:3A]  0.520 ミリ秒
3 プローブを送信 (1 ブロードキャスト)
3 応答を受信
```

出力 8: ndisc6(IPv6) の実行結果

```
guest@host82:~$ ndisc6 2001:2f8:40:11da::200 eth1
Soliciting 2001:2f8:40:11da::200 (2001:2f8:40:11da::200) on eth1...
Target link-layer address: FE:89:01:C2:83:3A
from 2001:2f8:40:11da::200
```

出力 9: 実行後の近隣キャッシュ

```
guest@host82:~$ ip ne
172.28.0.203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
172.28.0.200 dev eth1 lladdr fe:89:01:c2:83:3a STALE
172.28.0.1 dev eth1 lladdr a2:1e:33:b9:98:ef REACHABLE
fe80::fc89:1ff:fec2:833a dev eth1 lladdr fe:89:01:c2:83:3a REACHABLE
fe80::e4aa:7dff:fe51:5ced dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
fe80::a01e:33ff:feb9:98ef dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
2001:2f8:40:11da::1 dev eth1 lladdr a2:1e:33:b9:98:ef router DELAY
```

IPv4 では，arping により 172.28.0.200 の MAC アドレスが FE:89:01:C2:83:3A であると分かった．出力 7 には「3 プローブを送信 (1 ブロードキャスト)」とあり，最初の 1 回がブロードキャストの ARP Request で，残りの 2 回は MAC アドレスが分かった後のユニキャストであると読み取れる．図 3 では，No.272 の ARP Request の宛先が Broadcast(ff:ff:ff:ff:ff:ff) で，“Who has 172.28.0.200? Tell 172.28.0.182” という内

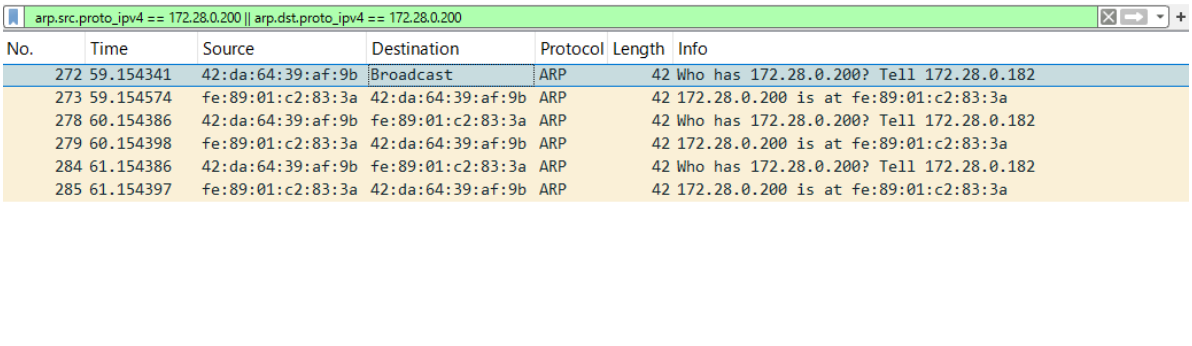



図 3: ARP のやり取りを Wireshark で読み取った結果

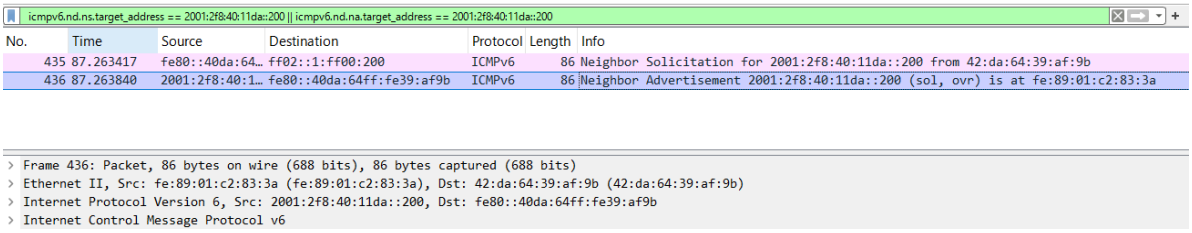



図 4: ND のやり取りを Wireshark で読み取った結果

容になっており，No.273 の ARP Reply では 172.28.0.200 が `fe:89:01:c2:83:3a` にいると返答している．続く No.278 と No.284 の Request は宛先が `fe:89:01:c2:83:3a` のユニキャストで，それぞれ No.279, No.285 で Reply が返っている．これは出力 7 の「1 ブロードキャスト」の内訳と一致する．

IPv6 では，ndisc6 により 2001:2f8:40:11da::200 の MAC アドレスが，IPv4 と同じ `FE:89:01:C2:83:3A` であると分かった．これは IPv4 と IPv6 の双方で，同一のホスト (sshd) の MAC アドレスが得られたことを示す．図 4 では，No.435 の Neighbor Solicitation(Type 135) が，宛先 `ff02::1:ff00:200`（2001:2f8:40:11da::200 に対する solicited-node マルチキャストアドレス）に送られている．その直後の No.436 の Neighbor Advertisement(Type 136) は，2001:2f8:40:11da::200 から要求元へユニキャストで返され，Target link-layer address として `fe:89:01:c2:83:3a` が通知されている．ARP がブロードキャストを使うのに対し，ND では対象のアドレスに対応するマルチキャストグループ宛てに Solicitation を送る点が異なる．

近隣キャッシュは，出力 6 と出力 9 を比べると，`fe80::fc89:1ff:fec2:833a` のエントリが新たに追加され，その MAC アドレスは上記と同じ `fe:89:01:c2:83:3a` であった．このアドレスは，MAC アドレスから EUI-64 形式で作られたリンクローカルアドレスである．一方，172.28.0.200 のエントリは実行前から存在し (STALE)，実行後も変化していなかった．それにもかかわらず，arping は No.272 でブロードキャストの Request を送信している．

# 課題 C

## 課題内容

LAN 内の TCP echo サーバに接続し，送信した文字列がそのまま返ってくることを，IPv4・IPv6 の双方で確認した．

## 実験方法

1. 1 つ目のターミナルでキャプチャを開始

```
tcpdump -i eth1 -s 0 -w C.pcap 'not port 4200'
```

2. キャプチャ中に，別のターミナルで telnet を実行し，適当な文字列を入力して返信を確認したのち，Ctrl+]，quit で終了する．IPv4 と IPv6 の双方で行う．

```
(IPv4) telnet 172.28.0.200 echo
(IPv6) telnet 2001:2f8:40:11da::200 echo
```

3. キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開く

## 結果

tcpdump の実行結果と，IPv4・IPv6 の telnet の実行結果をそれぞれ出力 10，出力 11，出力 12 に示した．

出力 10: tcpdump の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w C.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C500 packets captured
502 packets received by filter
0 packets dropped by kernel
```

出力 11: telnet(IPv4) の実行結果

```
guest@host82:~$ telnet 172.28.0.200 echo
Trying 172.28.0.200...
Connected to 172.28.0.200.
Escape character is '^]'.
hello
hello
world
world
^]
telnet> quit
Connection closed.
```

出力 12: telnet(IPv6) の実行結果

```
guest@host82:~$ telnet 2001:2f8:40:11da::200 echo
Trying 2001:2f8:40:11da::200...
Connected to 2001:2f8:40:11da::200.
Escape character is '^]'.
hoge
hoge
huga
huga
^]
telnet> quit
Connection closed.
```

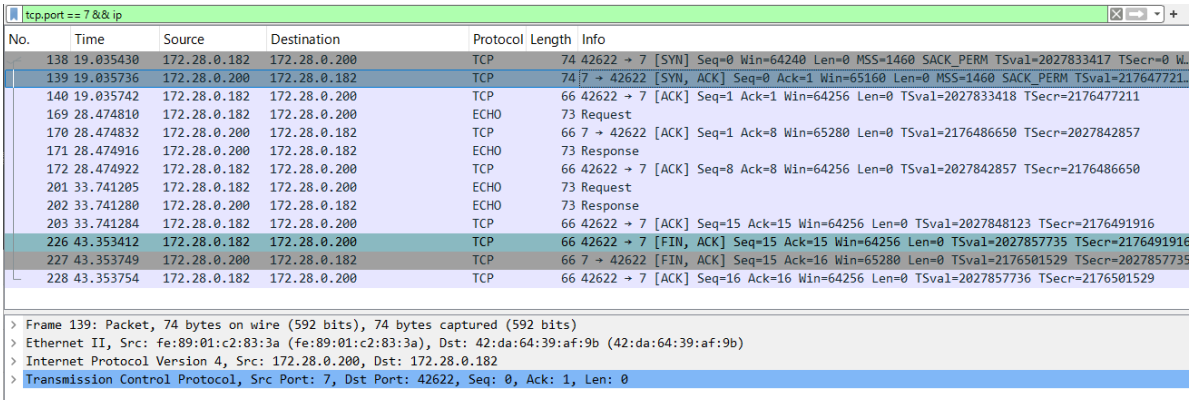



図 5: IPv4 の TCP echo 通信を Wireshark で読み取った結果

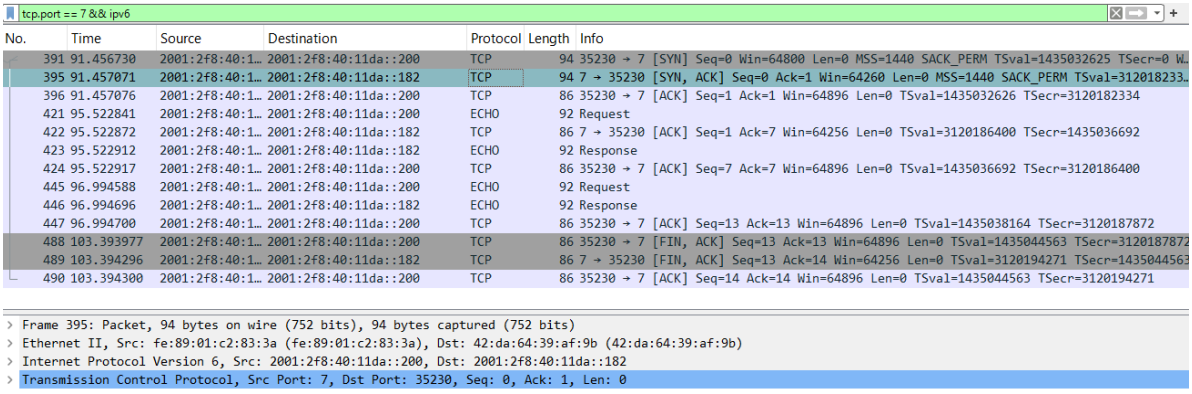



図 6: IPv6 の TCP echo 通信を Wireshark で読み取った結果

IPv4 では，出力 11 のとおり，入力した hello と world がそれぞれそのままサーバから返ってきた．図 5 では，まず No.138 の SYN，No.139 の SYN/ACK，No.140 の ACK により，TCP の 3 ウェイハンドシェイクで接続が確立している．続いて，No.169 で hello を含む ECHO の Request（データ長 7 バイト）がクライアント (172.28.0.182) からサーバ (172.28.0.200) へ送られ，サーバは No.170 で ACK を返したのち，No.171 で同じ 7 バイトの Response を返している．これをクライアントが No.172 で ACK している．world についても同様に，No.201 の Request，No.202 の Response，No.203 の ACK の順にやり取りされている．データ長が 7 バイトであるのは，hello の 5 バイトに，Enter で付加される改行コード (CR LF) の 2 バイトが加わっているためと考えられる．最後に，クライアントが quit で接続を終了した際，No.226 の FIN/ACK，No.227 の FIN/ACK，No.228 の ACK により接続が閉じられている．

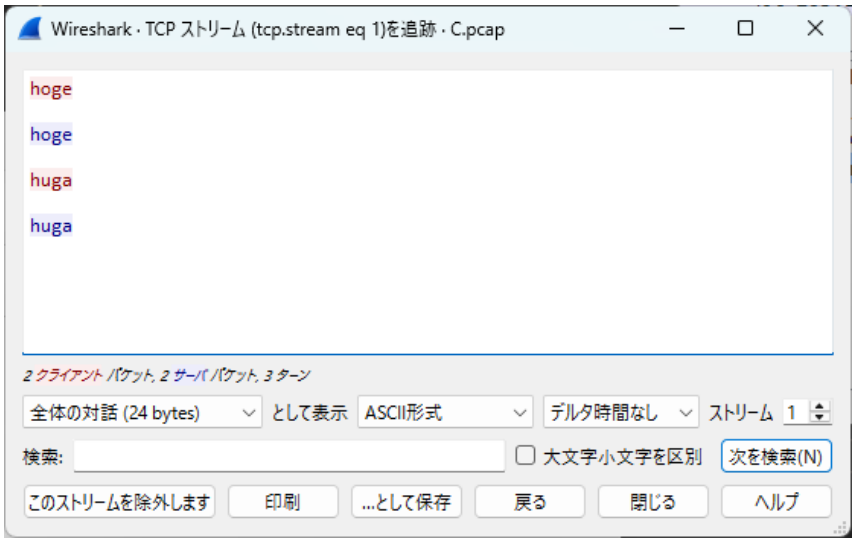



図 7: TCP ストリームの追跡結果 (IPv6)

IPv6 でも，出力 12 のとおり，入力した hoge と huga がそのまま返ってきた．図 6 では，クライアント (2001:2f8:40:11da::182) とサーバ (2001:2f8:40:11da::200) の間で，IPv4 と同じ手順で通信が行われている．No.391 の SYN，No.395 の SYN/ACK，No.396 の ACK で接続が確立し，No.421 の Request に対して No.422 の ACK，No.423 の Response（データはいずれも 6 バイト），No.424 の ACK が続く．huga についても，No.445 の Request，No.446 の Response，No.447 の ACK の順にやり取りされている．接続の終了は，No.488 の FIN/ACK，No.489 の FIN/ACK，No.490 の ACK である．図 7 の TCP ストリームの追跡 (IPv6) では，hoge と huga が送信側・受信側の双方で一致して表示され，全体の対話は 24 バイトであった．これは，6 バイトのデータが 4 回（送信 2 回，受信 2 回）やり取りされた合計と一致する．

IPv4 と IPv6 を比べると，TCP の手順（3 ウェイハンドシェイク，データの送受信，FIN による終了）は同じである．一方で，SYN パケットの長さは IPv4 の 74 バイトに対して IPv6 では 94 バイトと 20 バイト大きい．これは IP ヘッダが 20 バイトから 40 バイトになったためである．また，SYN に含まれる MSS の値も，IPv4 の 1460 に対して IPv6 では 1440 と 20 バイト小さく，同じ理由で説明できる．なお，echo サービスのポート番号は 7 であり，いずれの通信もサーバ側のポートは 7 である．

## 課題 D

### 課題内容

LAN 内の iperf3 サーバに対し，低速の UDP 通信を順方向・逆方向 (-R) の双方で行い，スループットとパケットの流れを確認した．

### 実験方法

1. 1 つ目のターミナルでキャプチャを開始

```
tcpdump -i eth1 -s 0 -w day2_D.pcap 'not port 4200'
```

2. キャプチャ中に，別のターミナルで iperf3 を順方向，逆方向の順に実行

```
（順方向）iperf3 -t 5 -u -b 10k -c 2001:2f8:40:11da::200
（逆方向）iperf3 -t 5 -R -u -b 10k -c 2001:2f8:40:11da::200
```

3. キャプチャ側で Ctrl-C を押し，pcap をダウンロードして Wireshark で開き，フィルタ `udp.port == 5201` を適用

## 結果

tcpdump の実行結果と，順方向・逆方向の iperf3 の実行結果をそれぞれ出力 13，出力 14，出力 15 に示した．

出力 13: tcpdump の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_D.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C520 packets captured
522 packets received by filter
0 packets dropped by kernel
```

出力 14: iperf3(順方向) の実行結果

```
guest@host82:~$ iperf3 -t 5 -u -b 10k -c 2001:2f8:40:11da::200
Connecting to host 2001:2f8:40:11da::200, port 5201
[  5] local 2001:2f8:40:11da::182 port 39751 connected to 2001:2f8:40:11da::200 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   1.00-2.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   2.00-3.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   3.00-4.00   sec  1.39 KBytes  11.4 Kbits/sec  1
[  5]   4.00-5.00   sec  1.39 KBytes  11.4 Kbits/sec  1
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-5.00   sec  6.97 KBytes  11.4 Kbits/sec  0.000 ms  0/5 (0%)  sender
[  5]   0.00-5.04   sec  6.97 KBytes  11.3 Kbits/sec  0.003 ms  0/5 (0%)  receiver

iperf Done.
```

出力 15: iperf3(逆方向) の実行結果

```
guest@host82:~$ iperf3 -t 5 -R -u -b 10k -c 2001:2f8:40:11da::200
Connecting to host 2001:2f8:40:11da::200, port 5201
Reverse mode, remote host 2001:2f8:40:11da::200 is sending
[  5] local 2001:2f8:40:11da::182 port 37219 connected to 2001:2f8:40:11da::200 port 5201
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-1.00   sec  1.39 KBytes  11.4 Kbits/sec  2.541 ms  0/1 (0%)
[  5]   1.00-2.00   sec  1.39 KBytes  11.4 Kbits/sec  4.921 ms  0/1 (0%)
[  5]   2.00-3.00   sec  1.39 KBytes  11.4 Kbits/sec  4.613 ms  0/1 (0%)
[  5]   3.00-4.00   sec  1.39 KBytes  11.4 Kbits/sec  4.325 ms  0/1 (0%)
[  5]   4.00-5.00   sec  1.39 KBytes  11.4 Kbits/sec  4.055 ms  0/1 (0%)
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-5.04   sec  6.97 KBytes  11.3 Kbits/sec  0.000 ms  0/5 (0%)  sender
[  5]   0.00-5.00   sec  6.97 KBytes  11.4 Kbits/sec  4.055 ms  0/5 (0%)  receiver

iperf Done.
```

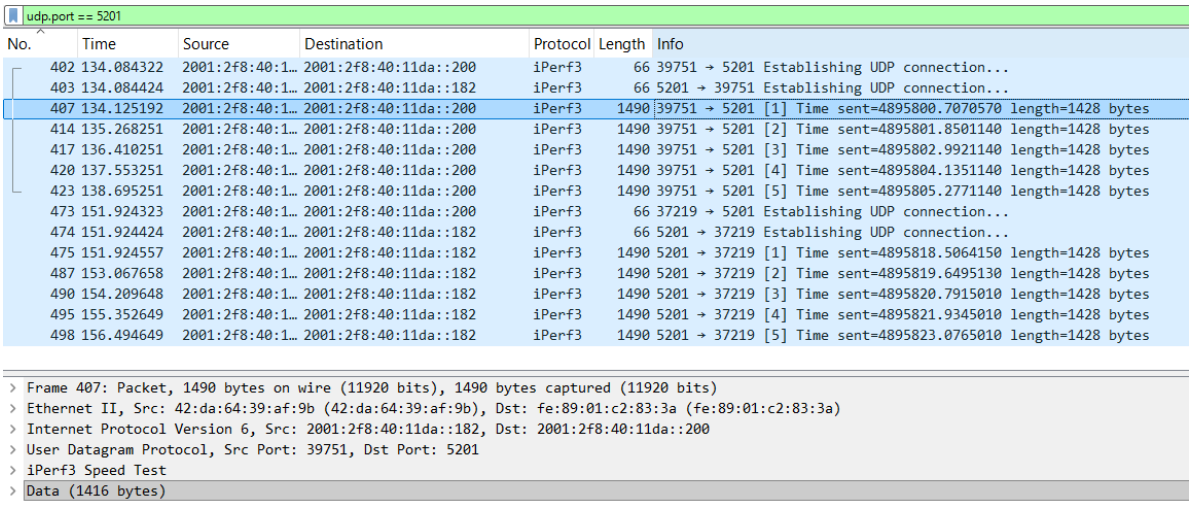



図 8: 順方向の UDP 通信を Wireshark で読み取った結果

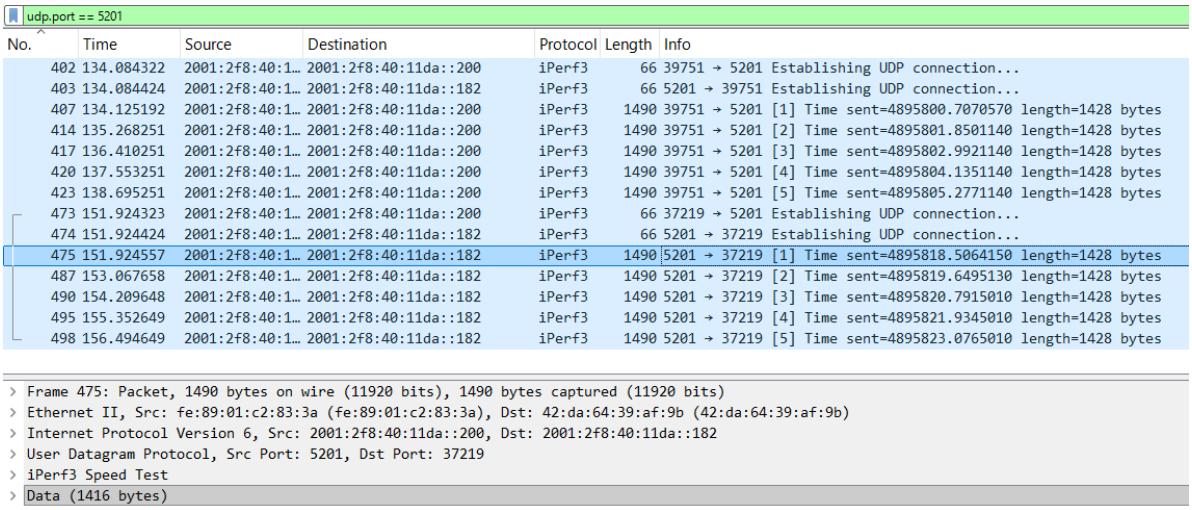



図 9: 逆方向の UDP 通信を Wireshark で読み取った結果

順方向では，出力 14 のとおり，指定した 10 kbit/s に対し実測の Bitrate は【要記入】であり，【要記

入】個のデータグラムを送信して損失は【要記入】であった．図 8 では，No.【要記入】でクライアント (2001:2f8:40:11da::182) からサーバ (2001:2f8:40:11da::200) の UDP 5201 番ポートへ短いデータグラム (【要記入】バイト) が送られ，No.【要記入】でサーバから応答が返っており，これが接続手続きに相当する．その後，No.【要記入】〜【要記入】でデータを含む UDP パケット (ペイロード長【要記入】バイト) が約【要記入】秒間隔で送られている．ペイロード長を $L$ バイトとすると，10 kbit/s を保つための送信間隔は $8L/10000$ 秒であり，$L =$【要記入】のとき【要記入】秒となって観測値と【一致する／しない】.

逆方向 (-R) では，データを送る側がサーバになる．図 9 では，接続手続きののち，サーバ (::200) からクライアント (::182) へデータを含む UDP パケットが送られており，パケットの向きが順方向と入れ替わっている．出力 15 の Bitrate は【要記入】，損失は【要記入】であった．

なお，フィルタ `udp.port == 5201` には，iperf3 の制御に使われる TCP の接続は含まれない．【要記入：TCP 5201 の制御接続を見て気づいたことがあれば】

## 課題 E

### 課題内容

学外の Web サイトに wget で HTTP/HTTPS アクセスし，通信の手順（ゲートウェイの MAC アドレス解決，DNS，HTTP，HTTPS）を確認した．

### 実験方法

1. 経路表と近隣キャッシュを記録

```
ip -4 route
ip -6 route
ip ne
```

2. 以下のサイトごとに，キャプチャ開始，wget 実行，キャプチャ停止を繰り返す（pcap はサイトごとに分ける）．tcpdump と wget は別のターミナルで実行し，wget の終了後に Ctrl-C でキャプチャを止める．

```
(IPv4 のみ)     tcpdump -i eth1 -s 0 -w day2_E_sapporo.pcap 'not port 4200'
                wget www.city.sapporo.jp
(IPv4/IPv6)     tcpdump -i eth1 -s 0 -w day2_E_hokkaido.pcap 'not port 4200'
                wget www.pref.hokkaido.lg.jp
(IPv6 のみ指定) tcpdump -i eth1 -s 0 -w day2_E_keio.pcap 'not port 4200'
                wget www.keio.ac.jp
```

3. 各 pcap を Wireshark で開き，次のフィルタを適用

```
(arp || icmpv6.type == 135 || icmpv6.type == 136 || dns || tcp.port == 80 || tcp.port == 443) &&
↪  !(dns.qry.type == 12)
```

## 結果

実行前の経路表と近隣キャッシュを出力 16 に，サイトごとの tcpdump と wget の実行結果を出力 17，出力 18，出力 19 に示した．

出力 16: 実行前の経路表と近隣キャッシュ

```
guest@host82:~$ ip -4 route
default via 172.28.0.1 dev eth1
172.27.0.0/24 dev eth0 proto kernel scope link src 172.27.0.182
172.28.0.0/24 dev eth1 proto kernel scope link src 172.28.0.182

guest@host82:~$ ip -6 route
2001:2f8:40:11da::/64 dev eth1 proto kernel metric 256 pref medium
fe80::/64 dev eth1 proto kernel metric 256 pref medium
default via 2001:2f8:40:11da::1 dev eth1 metric 1024 pref medium

guest@host82:~$ ip ne
172.28.0.203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
172.28.0.1 dev eth1 lladdr a2:1e:33:b9:98:ef REACHABLE
fe80::fc89:1ff:fec2:833a dev eth1 lladdr fe:89:01:c2:83:3a STALE
2001:2f8:40:11da::200 dev eth1 lladdr fe:89:01:c2:83:3a STALE
2001:2f8:40:11da::203 dev eth1 lladdr e6:aa:7d:51:5c:ed STALE
fe80::e4aa:7dff:fe51:5ced dev eth1 lladdr e6:aa:7d:51:5c:ed REACHABLE
fe80::a01e:33ff:feb9:98ef dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
2001:2f8:40:11da::1 dev eth1 lladdr a2:1e:33:b9:98:ef router REACHABLE
```

出力 17: www.city.sapporo.jp の tcpdump と wget の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_E_sapporo.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C152 packets captured
154 packets received by filter
0 packets dropped by kernel

guest@host82:~$ wget www.city.sapporo.jp
--2026-11-10 00:55:19--  http://www.city.sapporo.jp/
www.city.sapporo.jp (www.city.sapporo.jp) を DNS に問いあわせています... 45.60.112.77
www.city.sapporo.jp (www.city.sapporo.jp)|45.60.112.77|:80 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 301 Moved Permanently
場所: https://www.city.sapporo.jp/ [続く]
--2026-11-10 00:55:19--  https://www.city.sapporo.jp/
www.city.sapporo.jp (www.city.sapporo.jp)|45.60.112.77|:443 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 200 OK
長さ: 38349 (37K) [text/html]
`index.html' に保存中

index.html                100%[=====================================>]  37.45K  --.-KB/s    in 0.03s

2026-11-10 00:55:20 (1.27 MB/s) - `index.html' へ保存完了 [38349/38349]
```

出力 18: www.pref.hokkaido.lg.jp の tcpdump と wget の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_E_hokkaido.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C170 packets captured
172 packets received by filter
0 packets dropped by kernel

guest@host82:~$ wget www.pref.hokkaido.lg.jp
--2026-11-10 00:57:19--  http://www.pref.hokkaido.lg.jp/
www.pref.hokkaido.lg.jp (www.pref.hokkaido.lg.jp) を DNS に問いあわせています... 2a02:e980:10e::4d,
↪  45.60.112.77
www.pref.hokkaido.lg.jp (www.pref.hokkaido.lg.jp)|2a02:e980:10e::4d|:80 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 302 Found
場所: https://www.pref.hokkaido.lg.jp/ [続く]
--2026-11-10 00:57:19--  https://www.pref.hokkaido.lg.jp/
www.pref.hokkaido.lg.jp (www.pref.hokkaido.lg.jp)|2a02:e980:10e::4d|:443 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 200 OK
長さ: 特定できません [text/html]
`index.html.1' に保存中

index.html.1                    [ <=>                                   ]  93.46K  --.-KB/s    in 0.07s

2026-11-10 00:57:20 (1.40 MB/s) - `index.html.1' へ保存終了 [95704]
```

出力 19: www.keio.ac.jp の tcpdump と wget の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_E_keio.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C296 packets captured
298 packets received by filter
0 packets dropped by kernel

guest@host82:~$ wget www.keio.ac.jp
--2026-11-10 00:59:01--  http://www.keio.ac.jp/
www.keio.ac.jp (www.keio.ac.jp) を DNS に問いあわせています... 2600:9000:2142:5800:9:dcf4:5ec0:93a1,
↪  2600:9000:2142:50
00:9:dcf4:5ec0:93a1, 2600:9000:2142:3c00:9:dcf4:5ec0:93a1, ...
www.keio.ac.jp (www.keio.ac.jp)|2600:9000:2142:5800:9:dcf4:5ec0:93a1|:80 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 301 Moved Permanently
場所: https://www.keio.ac.jp/ [続く]
--2026-11-10 00:59:01--  https://www.keio.ac.jp/
www.keio.ac.jp (www.keio.ac.jp)|2600:9000:2142:5800:9:dcf4:5ec0:93a1|:443 に接続しています... 接続しました。
HTTP による接続要求を送信しました、応答を待っています... 301 Moved Permanently
場所: /ja/ [続く]
--2026-11-10 00:59:01--  https://www.keio.ac.jp/ja/
[www.keio.ac.jp]:443 への接続を再利用します。
HTTP による接続要求を送信しました、応答を待っています... 200 OK
長さ: 372926 (364K) [text/html]
`index.html.2' に保存中

index.html.2              100%[=====================================>] 364.19K  --.-KB/s    in 0.05s

2026-11-10 00:59:01 (6.60 MB/s) - `index.html.2' へ保存完了 [372926/372926]
```

学外への通信は，まずデフォルトゲートウェイの MAC アドレスを解決することから始まる．出力 16 の経路表では，デフォルトゲートウェイは IPv4 が【要記入】，IPv6 が【要記入】であり，近隣キャッシュには両者の MAC アドレス【要記入】がすでに登録されていた．そのため，図 10〜図 12 では【ARP/ND のパケットが現れなかった／現れた】．【現れなかった場合：キャッシュの内容 (出力 16) から，ゲートウェイの MAC アドレスは既知であったと言える．】

www.city.sapporo.jp では，図 10 の No.【要記入】で名前解決のため A レコードの問い合わせが DNS リゾルバ【要記入】に送られ，No.【要記入】の応答でアドレス【要記入】が得られた．AAAA レコードの問い合わせ (No.【要記入】) に対する応答は【アドレスなし】であり，このサイトは IPv4 のみに対応していることが分かる．続いて，TCP の接続後に HTTP の GET がポート 80 に送られ，No.【要記入】のレスポンスは【要記入：301 などのステータスコードと Location】であった．【HTTPS へ転送されている場合：その後ポート 443 へ接続し，TLS ハンドシェイク (Client Hello，Server Hello，Certificate 等) ののち HTTPS でコンテンツが取得された．】

www.pref.hokkaido.lg.jp は，図 11 のとおり A と AAAA の双方に応答があり，IPv4/IPv6 の両方に対応している．wget が実際に使用したのは【IPv4 ／ IPv6】で，その理由は【要記入】と考えられる．HTTP および HTTPS の手順は【要記入】．

www.keio.ac.jp については，図 12 で【要記入：問い合わせるレコード，使用した IP バージョン，HTTP/HTTPS の手順】．

fig/wireshark_e_sapporo.png

図 10: www.city.sapporo.jp へのアクセスを Wireshark で読み取った結果

以上より，【3 サイトの違い（IP バージョン，リダイレクトの有無，TLS のバージョンなど）を 2〜3 文でまとめる】.

## 課題 F

### 課題内容

HTTP/3 対応の Web サイトに curl でアクセスし，通信をキャプチャして HTTP/2 と比較した.

### 実験方法

1. HTTP/3 でアクセスし，ヘッダを取得

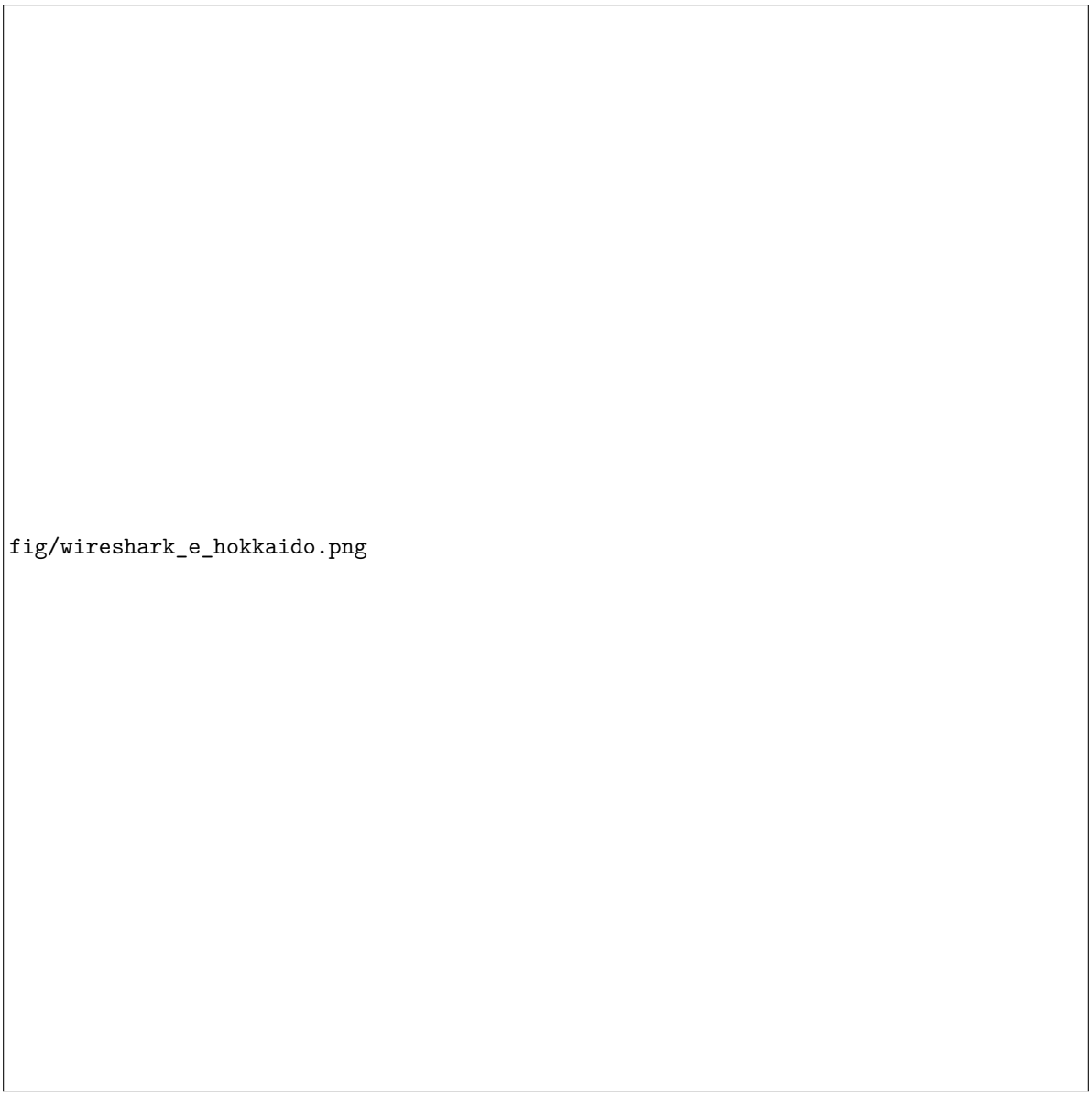



図 11: www.pref.hokkaido.lg.jp へのアクセスを Wireshark で読み取った結果

```
tcpdump -i eth1 -s 0 -w day2_F_h3.pcap 'not port 4200'    (1 つ目のターミナル)
curl -I --http3 https://www.cloudflare.com                (2 つ目のターミナル)
```

2. 比較用に HTTP/2 でアクセス

```
tcpdump -i eth1 -s 0 -w day2_F_h2.pcap 'not port 4200'    (1 つ目のターミナル)
curl -I --http2 https://www.cloudflare.com                (2 つ目のターミナル)
```

3. 表にない HTTP/3 対応サイトの探索．alt-svc ヘッダに h3 が含まれるサイトを探す

```
curl -sI https://<候補の URL> | grep -i alt-svc
```

4. 各 pcap を Wireshark で開き，次のフィルタを適用

```
(HTTP/3) quic
(HTTP/2) tcp.port == 443
```

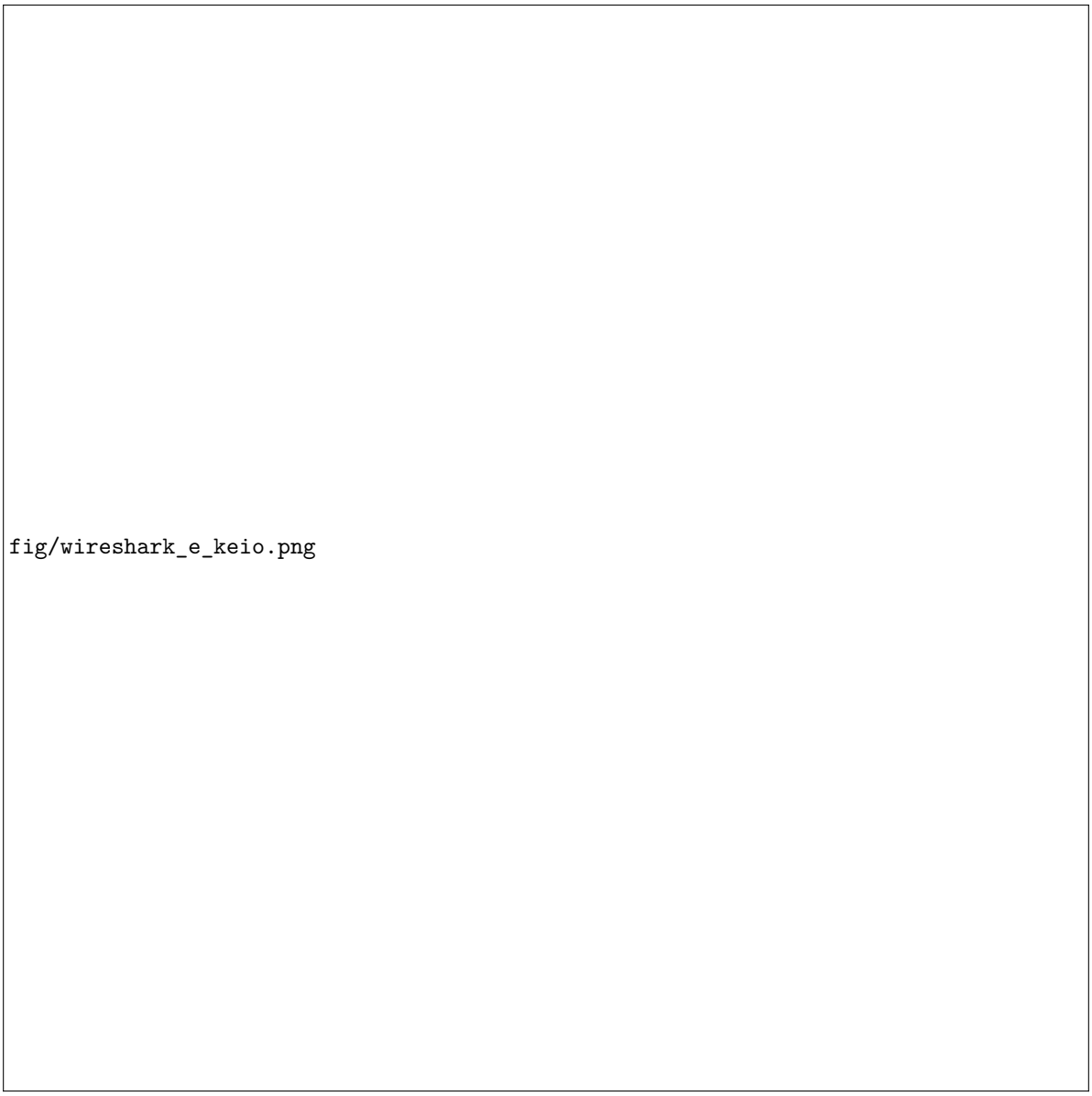



図 12: www.keio.ac.jp へのアクセスを Wireshark で読み取った結果

### 結果

HTTP/3・HTTP/2 それぞれの tcpdump と curl の実行結果を出力 20，出力 21 に，alt-svc の探索結果を出力 22 に示した.

出力 20: HTTP/3 の tcpdump と curl の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_F_h3.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C213 packets captured
215 packets received by filter
0 packets dropped by kernel

guest@host82:~$ curl -I --http3 https://www.cloudflare.com
HTTP/3 103
link: </fonts/Kunst%20Grotesk%20Regular.woff2>; as=font; crossorigin; rel=preload; type=font/woff2,
</fonts/Kunst%20Grotesk%20Medium.woff2>; as=font; crossorigin; rel=preload; type=font/woff2, <https:
//ot.www.cloudflare.com>; crossorigin; rel=preconnect, <https://imagedelivery.net>; crossorigin; rel
=preconnect, </static/hero-poster.avif>; as=image; fetchpriority=high; rel=preload; type=image/avif

HTTP/3 200
date: Mon, 09 Nov 2026 16:22:59 GMT
content-type: text/html; charset=utf-8
link: <https://www.cloudflare.com/.well-known/api-catalog>; rel="api-catalog", <https://www.cloudfla
re.com/.well-known/agents.json>; rel="service-desc", <https://www.cloudflare.com/.well-known/webmcp.
json>; rel="service-desc", <https://www.cloudflare.com/openapi.json>; rel="service-desc", <https://w
ww.cloudflare.com/llms.txt>; rel="service-doc", <https://www.cloudflare.com/sitemap.xml>; rel="sitem
ap", </fonts/Kunst%20Grotesk%20Regular.woff2>; rel=preload; as=font; type="font/woff2"; crossorigin,
</fonts/Kunst%20Grotesk%20Medium.woff2>; rel=preload; as=font; type="font/woff2"; crossorigin, <htt
ps://ot.www.cloudflare.com>; rel=preconnect; crossorigin, <https://imagedelivery.net>; rel=preconnec
t; crossorigin, </static/hero-poster.avif>; rel=preload; as=image; type="image/avif"; fetchpriority=
high
cache-control: no-store
server: cloudflare
vary: accept-encoding
strict-transport-security: max-age=31536000; includeSubDomains
content-security-policy: default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval' https:/
/static.cloudflareinsights.com https://static-staging.cloudflareinsights.com https://challenges.clou
dflare.com https://*.onetrust.com https://cdn.cookielaw.org https://ot.www.cloudflare.com https://ww
w.googletagmanager.com https://tagmanager.google.com https://www.googleadservices.com https://google
ads.g.doubleclick.net https://adservice.google.com https://cdn.bizible.com https://js.adsrvr.org htt
ps://*.marketo.net https://platform.twitter.com https://static.ads-twitter.com https://scripts.deman
dbase.com https://tag.demandbase.com https://*.6sc.co https://*.qualified.com https://snap.licdn.com
https://bat.bing.com; style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; font-src 'self
' data:; connect-src 'self' https://*.googletagmanager.com https://*.google-analytics.com https://*.
google.com https://*.doubleclick.net https://www.googleadservices.com https://translate.googleapis.c
om https://cdn.bizible.com https://js.adsrvr.org https://*.marketo.net https://ads-twitter.com https
://analytics.twitter.com https://*.twimg.com https://api.demandbase.com https://scripts.demandbase.c
om https://tag.demandbase.com https://tag-logger.demandbase.com https://api.company-target.com https
://*.6sc.co https://epsilon.6sense.com https://*.qualified.com https://*.ads.linkedin.com https://ww
w.linkedin.com https://bat.bing.com https: wss://*.qualified.com wss://*.registrar.cloudflare.com; f
rame-src https://*.adsrvr.org https://*.cloudflare.com https://*.videodelivery.net https://*.cloudfl
arestream.com https://www.googletagmanager.com https://*.qualified.com https://td.doubleclick.net ht
tps://bid.g.doubleclick.net https://9309168.fls.doubleclick.net https://9973066.fls.doubleclick.net
https://s.company-target.com; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestor
s 'none'; worker-src 'self' blob:; child-src 'self' blob:; upgrade-insecure-requests
cross-origin-opener-policy: unsafe-none
cross-origin-resource-policy: cross-origin
permissions-policy: geolocation=(), camera=(), microphone=()
referrer-policy: strict-origin-when-cross-origin
guest@host82:~$ curl -I --http3 https://www.cloudflare.com
HTTP/3 103
link: </fonts/Kunst%20Grotesk%20Regular.woff2>; as=font; crossorigin; rel=preload; type=font/woff2,
</fonts/Kunst%20Grotesk%20Medium.woff2>; as=font; crossorigin; rel=preload; type=font/woff2, <https:
//ot.www.cloudflare.com>; crossorigin; rel=preconnect, <https://imagedelivery.net>; crossorigin; rel
=preconnect, </static/hero-poster.avif>; as=image; fetchpriority=high; rel=preload; type=image/avif
```

出力 21: HTTP/2 の tcpdump と curl の実行結果

```
guest@host82:~$ tcpdump -i eth1 -s 0 -w day2_F_h2.pcap 'not port 4200'
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
^C112 packets captured
114 packets received by filter
0 packets dropped by kernel

guest@host82:~$ curl -I --http2 https://www.cloudflare.com
HTTP/2 103
link: </fonts/Kunst%20Grotesk%20Regular.woff2>; as=font; crossorigin; rel=preload; type=font/woff2,
</fonts/Kunst%20Grotesk%20Medium.woff2>; as=font; crossorigin; rel=preload; type=font/woff2, <https:
//ot.www.cloudflare.com>; crossorigin; rel=preconnect, <https://imagedelivery.net>; crossorigin; rel
=preconnect, </static/hero-poster.avif>; as=image; fetchpriority=high; rel=preload; type=image/avif

HTTP/2 200
date: Mon, 09 Nov 2026 16:26:18 GMT
content-type: text/html; charset=utf-8
link: <https://www.cloudflare.com/.well-known/api-catalog>; rel="api-catalog", <https://www.cloudfla
re.com/.well-known/agents.json>; rel="service-desc", <https://www.cloudflare.com/.well-known/webmcp.
json>; rel="service-desc", <https://www.cloudflare.com/openapi.json>; rel="service-desc", <https://w
ww.cloudflare.com/llms.txt>; rel="service-doc", <https://www.cloudflare.com/sitemap.xml>; rel="sitem
ap", </fonts/Kunst%20Grotesk%20Regular.woff2>; rel=preload; as=font; type="font/woff2"; crossorigin,
</fonts/Kunst%20Grotesk%20Medium.woff2>; rel=preload; as=font; type="font/woff2"; crossorigin, <htt
ps://ot.www.cloudflare.com>; rel=preconnect; crossorigin, <https://imagedelivery.net>; rel=preconnec
t; crossorigin, </static/hero-poster.avif>; rel=preload; as=image; type="image/avif"; fetchpriority=
high
cache-control: no-store
vary: accept-encoding
strict-transport-security: max-age=31536000; includeSubDomains
content-security-policy: default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval' https:/
/static.cloudflareinsights.com https://static-staging.cloudflareinsights.com https://challenges.clou
dflare.com https://*.onetrust.com https://cdn.cookielaw.org https://ot.www.cloudflare.com https://ww
w.googletagmanager.com https://tagmanager.google.com https://www.googleadservices.com https://google
ads.g.doubleclick.net https://adservice.google.com https://cdn.bizible.com https://js.adsrvr.org htt
ps://*.marketo.net https://platform.twitter.com https://static.ads-twitter.com https://scripts.deman
dbase.com https://tag.demandbase.com https://*.6sc.co https://*.qualified.com https://snap.licdn.com
https://bat.bing.com; style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; font-src 'self
' data:; connect-src 'self' https://*.googletagmanager.com https://*.google-analytics.com https://*.
google.com https://*.doubleclick.net https://www.googleadservices.com https://translate.googleapis.c
om https://cdn.bizible.com https://js.adsrvr.org https://*.marketo.net https://ads-twitter.com https
://analytics.twitter.com https://*.twimg.com https://api.demandbase.com https://scripts.demandbase.c
om https://tag.demandbase.com https://tag-logger.demandbase.com https://api.company-target.com https
://*.6sc.co https://epsilon.6sense.com https://*.qualified.com https://*.ads.linkedin.com https://ww
w.linkedin.com https://bat.bing.com https: wss://*.qualified.com wss://*.registrar.cloudflare.com; f
rame-src https://*.adsrvr.org https://*.cloudflare.com https://*.videodelivery.net https://*.cloudfl
arestream.com https://www.googletagmanager.com https://*.qualified.com https://td.doubleclick.net ht
tps://bid.g.doubleclick.net https://9309168.fls.doubleclick.net https://9973066.fls.doubleclick.net
https://s.company-target.com; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestor
s 'none'; worker-src 'self' blob:; child-src 'self' blob:; upgrade-insecure-requests
cross-origin-opener-policy: unsafe-none
cross-origin-resource-policy: cross-origin
permissions-policy: geolocation=(), camera=(), microphone=()
referrer-policy: strict-origin-when-cross-origin
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
x-served-by: marketing-site
x-xss-protection: 1; mode=block
x-rm: GW
set-cookie: _ga=GA1.1.ba1f85ac-4496-4599-9622-828b7af17f09; Expires=Thu, 09 Dec 2027 16:26:18 GMT; P
ath=/
```

出力 22: alt-svc ヘッダの探索結果

```
guest@host82:~$ curl -sI https://【候補の URL】| grep -i alt-svc
% TODO: 実際の出力を貼る
```

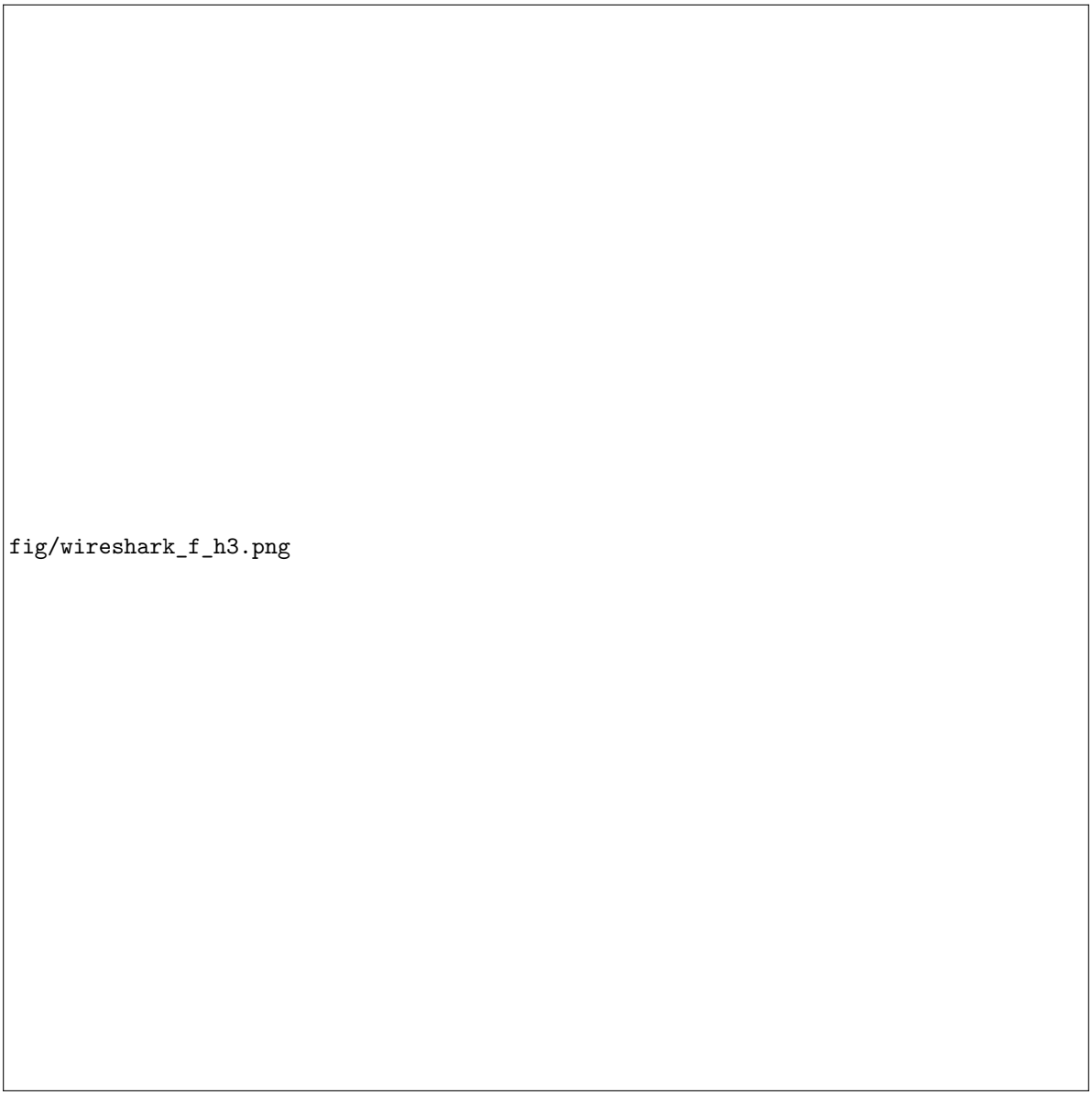



図 13: HTTP/3(QUIC) の Initial パケットを Wireshark で読み取った結果

HTTP/3 では，出力 20 のとおり【要記入：ステータスラインと，HTTP/3 で応答したことが分かる部分】. 図 13 では，クライアントからサーバの UDP 443 番ポートへ送られた Initial パケット (No.【要記入】，長さ【要記入】バイト) を展開した．Long Header には，Version(【要記入】)，Destination Connection ID(【要記入】)，Source Connection ID(【要記入】) が含まれ，ペイロードの CRYPTO フレームには TLS の Client Hello【要記入：SNI，ALPN の h3 など】が入っている．つまり QUIC では，TLS 1.3 のハンドシェイクが QUIC のパケット内で運ばれ，TCP のような独立した接続確立の手順は存在しない.

HTTP/2 では，図 14 のとおり，No.【要記入】の SYN，No.【要記入】の SYN/ACK，No.【要記入】の ACK で TCP の 3 ウェイハンドシェイクを行ったのち，Client Hello(No.【要記入】) から始まる TLS のハンドシェ

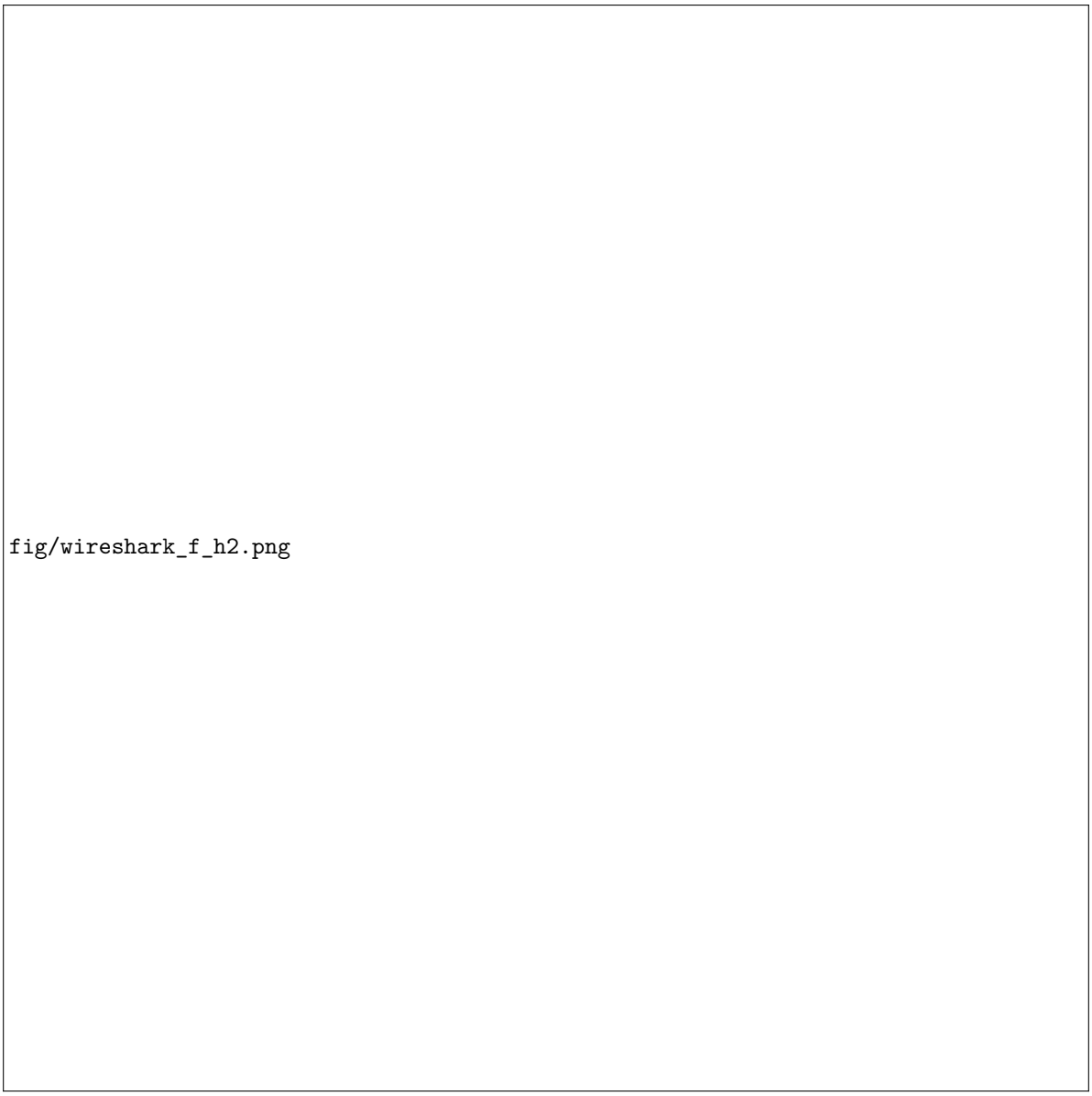



図 14: HTTP/2 の TCP ハンドシェイクと TLS を Wireshark で読み取った結果

イクが行われている．Client Hello の ALPN には【要記入】が含まれていた．

両者を比べると，HTTP/2 では TCP の接続確立と TLS のハンドシェイクが別々に必要であるのに対し，HTTP/3 では QUIC の最初のやり取りに暗号化の交渉が含まれる．実際に TCP の SYN から最初の HTTP リクエストまでに要した時間は【要記入】秒，QUIC では【要記入】秒であった．【時間差が出た／出なかった理由を一言】．

alt-svc ヘッダの探索では，出力 22 のとおり，【候補の URL】の alt-svc に【h3=":443" など】が含まれており，HTTP/3 に対応していることが分かった．alt-svc は，同じサイトが別のプロトコル (ここでは HTTP/3) でも提供されていることをクライアントに伝えるヘッダである．